Mechanistic Interpretability, Variables, and the Importance of Interpretable Bases

An informal note on some intuitions related to Mechanistic Interpretability by Chris Olah.

Mechanistic interpretability seeks to reverse engineer neural networks, similar to how one might reverse engineer a compiled binary computer program. After all, neural network parameters are in some sense a binary computer program which runs on one of the exotic virtual machines we call a neural network architecture.

This is actually quite a deep analogy. We'll discuss it more as this essay unfolds, but some parallels are listed in the table below:

Regular Computer Programs	**Neural Networks**
Reverse Engineering	Mechanistic Interpretability
Program Binary	Network Parameters
VM / Processor / Interpreter	Network Architecture
Program State / Memory	Layer Representation / Activations
Variable / Memory Location	Neuron / Feature Direction

Taking this analogy seriously can let us explore some of the big picture questions in mechanistic interpretability. Often, questions that feel speculative and slippery for reverse engineering neural networks become clear if you pose the same question for reverse engineering of regular computer programs. And it seems like many of these answers plausibly transfer back over to the neural network case.

Perhaps the most interesting observation is that this analogy seems to suggest that finding and understanding interpretable neurons – analogous to understanding variables in a computer program – isn't just one of many interesting questions. Arguably, it's the central task.

Attacking the Curse of Dimensionality

Every approach to interpretability must somehow overcome the curse of dimensionality. Neural networks are functions which typically have extremely high-dimensional input spaces. The n-dimensional volume of the input space grows exponentially as the number of dimensions increases, making it incredibly large. This is the *curse of dimensionality*. It is normally brought up as a challenge to learning functions: how can we learn a function over such a large input space without an exponential amount of data? But it's also a challenge for interpretability: how can we hope to understand a function over such a large space, without an exponential amount of time?[1]

One answer is to study toy neural networks with low-dimensional inputs, allowing easy full understanding by dodging the problem. Another answer is to study the behavior of neural networks in a neighborhood around an individual data point of interest – this is roughly the answer of saliency maps.

How does mechanistic interpretability solve the curse of dimensionality? It's worth asking this question of regular reverse engineering as well! Somehow, a programmer reverse engineering a computer program is able to understand its behavior, often over an incredibly high-dimensional space of inputs. They are able to do this **because the code gives a non-exponential description of the program's behavior**, which is an alternative to understanding the program as a function over a huge input space. We can aim for this same answer in the context of artificial neural networks. Ultimately, the parameters are a finite description of a neural network, if we can somehow understand them.

Of course, the parameters may be very large – hundreds of billions of parameters for the largest language models! But binary computer programs like a compiled operating system can also be very large, and we're often able to eventually understand them.

Another consequence is that we shouldn't expect mechanistic interpretability to be easy or have a cookie cutter process that can be followed. People often want interpretability to provide simple answers, a short explanation. But we should expect mechanistic interpretability to be at least as difficult as reverse engineering a large, complicated computer program.

Variables & Activations

Understanding a computer program requires us to both understand variables and understand operations acting on those variables. A statement like `y = x + 5;` is meaningless unless one understands what `y` and `x` are. At the same time, ultimately the meaning of `y` and `x` comes from how they're used by operations somewhere in the program. Presumably this is why variable names are so useful!

Reverse engineers generally don't have the benefit of variable names. They need to figure out what each variable actually represents. In fact, that understates the problem. Programs actually act on a collection of computer memory – their *state* – which humans think about in terms of discrete variables. In many cases, it's obvious how memory maps to variables, but this isn't always the case. So a reverse engineer must figure out how to segment the memory into variables which can be understood separately, and then what the meaning of those variables is. Put another way, a reverse engineer must assign meaning to positions in memory.

Reverse engineering neural networks faces an almost identical challenge. As discussed by Voss *et al*, neural network parameters might be thought of as binary instructions, while neuron activations are analogous to variables or memory. Each parameter describes how previous activations affect later activations. We can only understand the meaning of a parameter if we understand the input and output activations.

But the activations are high dimensional vectors: how can we hope to understand them? We've run back into the curse of dimensionality, but again, regular reverse engineering points at a solution. It's possible to understand computer program memory – also a high-dimensional space! – because we can segment it into variables which can be reasoned about and understood separately. Similarly, we need to break neural network activations into independently understandable pieces.

In some very special cases, including attention-only transformers (see Elhage *et al*), we can use linearity to describe all the network's operations in terms of the inputs and outputs of the model. This is similar to how some functions in a computer program can be described solely in terms of the arguments and return value, without intermediate variables. Since we generally understand the inputs and outputs, this allows us to side step the problem. But in most cases, we can't do this – what then?

If we can't avoid the problem with special tricks, mechanistic interpretability requires that we *must* somehow decompose activations into independently understandable pieces.

Simple Memory Layout & Neurons

Computer programs often have memory layouts that are convenient to understand. For example, most bytes in memory represent a single thing, rather than having each bit represent something unrelated. This is partly because these layouts are easier for programmers to think about, but it's also because our hardware often makes "simple" contiguous memory layouts more efficient. Having these simple memory layouts makes it easier to reverse engineer computer programs. It makes it easier to figure out how one should break memory up into independently understandable variables.

Something kind of analogous to this often happens in neural networks.

Let's assume for a moment that neural networks can be understood in terms of operations on a collection of independent "interpretable features". In principle, one could imagine "interpretable features" being embedded as arbitrary directions in activation space. But often, it seems like neural network layers with activation functions align features with their basis dimensions. This is because the activation functions in some sense make these directions natural and useful. Just as a CPU having operations that act on bytes encourages information to be grouped in bytes rather than randomly scattered over memory, activation functions often encourage features to be aligned with a neuron, rather than correspond to a random linear combination of neurons. We call this a *privileged basis*.

Having features align with neurons would make neural networks much easier to reverse engineer. This isn't to say that a neural network is impossible to reverse engineer without neurons being individually understandable. But it seems much harder, just as it is harder to reverse engineer computer programs with strange memory layouts.

Unfortunately, many neurons can't be understood this way. These *polysemantic neurons* seem to help represent features which are not best understood in terms of individual neurons. This is a really tricky problem for reverse engineering neural networks, which we discuss more in the SoLU paper (see especially Section 3).

For now, the main point we wish to make is that the ability to decompose representations into independently understandable parts seems essential for the success of mechanistic interpretability.

Learn More

This essay is a very informal discussion of some of the intuitions that motivate me in personally working on mechanistic interpretability. If you found it interesting and want to learn more about this line of research, check out the original Circuits thread and the new Transformer Circuits thread.

Random Related Topics FAQ

Doesn't having interpretable neurons go against having a distributed representation?

That isn't how we understand the term "distributed representation" in the context of Deep Learning. For example, a word embedding might have dimensions corresponding to interpretable features such as singular-plural, male-female, and large-small. One could rotate it so that those are basis dimensions. The thing that makes it "distributed" is that independent aspects of words are represented by different directions in representation space.

What do you mean by "feature"?

Defining a "feature" in a satisfying way is surprisingly hard. This isn't necessarily a bad thing. Sometimes when you're trying to understand something, you need to struggle with lots of definitions. (Lakatos' famous *Proofs and Refutations* is a beautiful articulation of this.)

The easy answer would be to say *a feature is something like curve detector or car detector neurons found in InceptionV1 – some meaningful, articulable property of the input which the network encodes as a direction in activation space.* The unsatisfactory thing about this definition is that it's human centric: it excludes the possibility of features humans don't understand.

An alternative definition, which I find more satisfactory, is to say that *features are whatever the "independent units" a neural network representation can be decomposed into are*. That definition is a bit fuzzier, but I suspect it might be getting at something more fundamental. Another idea for a potential human-independent definition is that *features are the things a network would ideally dedicate a neuron to if you gave it enough neurons*.

Some researchers define features simply as any function of the input. The thing I find unsatisfactory about this is that under this definition, any mixture of features is also a feature. It seems to me that there's something importantly different about a "car detector" (which seems to correspond to an important latent variable) from "car and cat detector" (which seems to be an arbitrary mixture of things). I'd like for a definition of a feature to make this distinction.

A lot of people seem to think that modularity is the key thing for understanding neural networks, but the circuits work keeps focusing on whether neurons can be understood. Why the difference?

The brief answer is that both seem very helpful – and we're hopeful they can be linked, as in branch specialization – but it seems to us that being able to decompose into understandable features *is what makes mechanistic interpretability possible at all* while modularity *may make it easier once it is possible*.

The intuitive focus on modularity seems very natural. In day-to-day programming, modularity is often the difference between code being easy or hard to understand. Another intuition is that the only way we'll be able to understand neural networks is if we can break them down into small enough chunks to understand, which seems like a kind of modularity. However, while modularity helps in day to day programming, we think it isn't the thing that makes it possible to understand programs at all. Understanding a computer program written as one giant function might be difficult, but trying to understand a program where memory isn't broken down into variables with coherent meanings (let alone variable names) seems terrifying!

In some ways, interpretable features create the ultimate kind of modularity: the network can be broken down into individual parameters, since each parameter can be thought of independently as connecting two features.

Publication

Published June 27, 2022 by Chris Olah.

Acknowledgments

The ideas in this note are based on many conversations with others, especially Nelson Elhage, Catherine Olsson, Tristan Hume, Dario Amodei, Chelsea Voss, Nick Cammarata, Gabriel Goh, Neel Nanda, Martin Wattenberg, Shan Carter, and Ludwig Schubert. The note itself was greatly improved by the feedback of Nelson Elhage and Martin Wattenberg.

Footnotes

A possible objection is that, while the input space is high-dimensional, we only need to understand the behavior of the function over the data manifold of actual inputs. (I was, at one point, personally quite invested in this approach!) However, while the data manifold is certainly lower dimensional than the input space but for tasks we care about like vision or language, it seems like it must still be very high-dimensional. For example, for any given natural image, it seems like one could move along the data manifold by having an arbitrary object enter the image from any side of the field of view – that's a lot of dimensions! [↩]